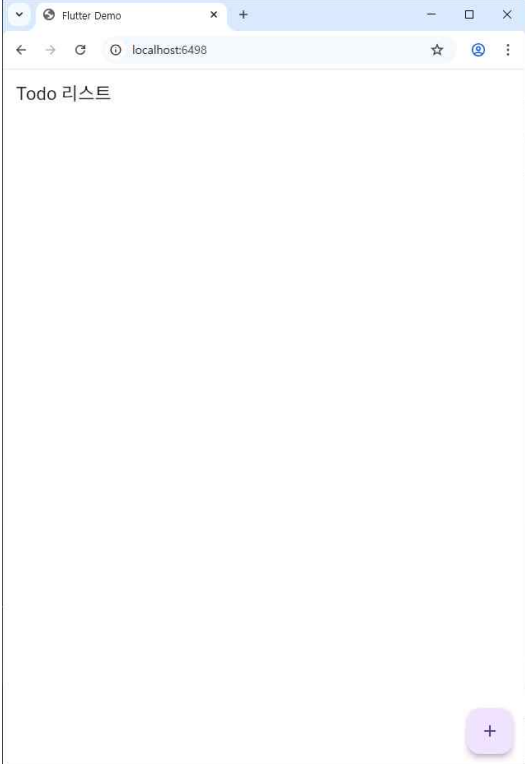
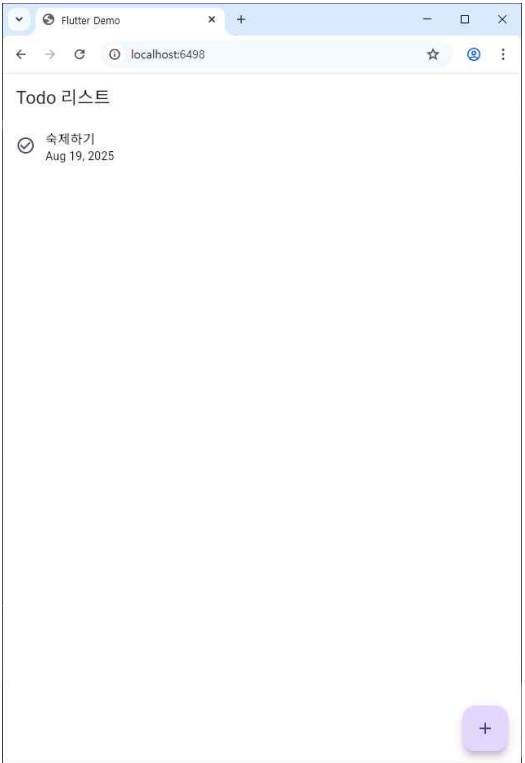
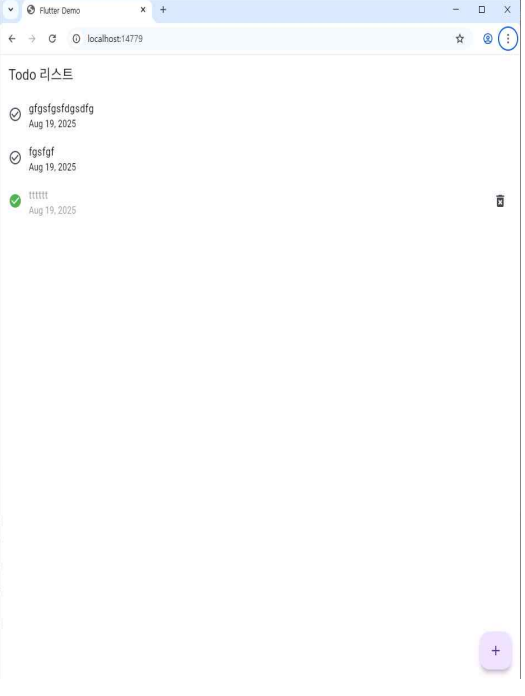
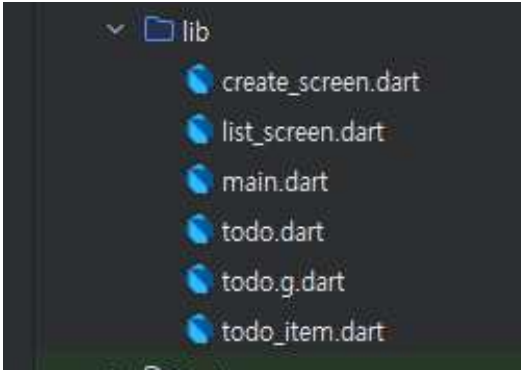


프로젝트 이름 : todolist_30000

실행1 	2 (create_screen) 
3 (list_screen) + 4 (todo_item)	
	

프로젝트 구성	pubspec.yaml
	<pre>dependencies: flutter: sdk: flutter cupertino_icons: ^1.0.2 hive_flutter: ^1.1.0 hive: ^2.2.3 intl: ^0.17.0 dev_dependencies: flutter_test: sdk: flutter flutter_lints: ^5.0.0 hive_generator: ^2.0.1 build_runner: ^2.4.15</pre>

main.dart -----

```
import 'package:flutter/material.dart';
import 'package:hive_flutter/hive_flutter.dart';    //간단한 내용 저장 패키지

import 'list_screen.dart';
import 'todo.dart';

late final Box<Todo> todos;

void main() async {
  await Hive.initFlutter();
  Hive.registerAdapter(TodoAdapter());
  todos = await Hive.openBox<Todo>('todolist.db');

  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // This widget is the root of your application.
```

```

@override
Widget build(BuildContext context) {
  return MaterialApp(
    debugShowCheckedModeBanner: false,
    title: 'Flutter Demo',
    theme: ThemeData(
      primarySwatch: Colors.blue,
    ),
    home: const ListScreen(),
  );
}
}

```

[create_screen.dart](#) -----

```

import 'package:flutter/material.dart';
import 'package:todolist/todo.dart';

import '../main.dart';

class CreateScreen extends StatefulWidget {
  const CreateScreen({super.key});

  @override
  State<CreateScreen> createState() => _CreateScreenState();
}

class _CreateScreenState extends State<CreateScreen> {
  final _textController = TextEditingController();

  @override
  void dispose() {
    _textController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(

```

```

title: const Text('Todo 작성'),
actions: [
  IconButton(
    onPressed: () async {
      await todos.add(Todo(
        title: _textController.text,
        dateTime: DateTime.now().millisecondsSinceEpoch,
      ));

      if (mounted) {
        Navigator.pop(context);
      }
    },
    icon: const Icon(Icons.done),
  ),
],
),
body: Padding(
  padding: const EdgeInsets.all(8.0),
  child: TextField(
    controller: _textController,
    decoration: InputDecoration(
      border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(10.0),
      ),
      filled: true,
      hintStyle: TextStyle(color: Colors.grey[800]),
      hintText: "할일을 입력하세요",
      fillColor: Colors.white70),
    ),
  ),
);
}
}

```

[list_screen.dart](#) -----

```
import 'package:flutter/material.dart';
```

```
import 'create_screen.dart';
```

```

import 'main.dart';
import 'todo_item.dart';

class ListScreen extends StatefulWidget {
  const ListScreen({super.key});

  @override
  State<ListScreen> createState() => _ListScreenState();
}

class _ListScreenState extends State<ListScreen> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Todo 리스트'),
      ),
      body: ListView(
        children: todos.values
          .map(
            (todo) => TodoItem(
              todo: todo,
              onTap: (todo) async {
                todo.isDone = !todo.isDone;
                await todo.save();

                setState(() {});
              },
              onDelete: (todo) async {
                await todo.delete();

                setState(() {});
              },
            ),
          )
          .toList(),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () async {
          await Navigator.push(
            context,

```

```

        MaterialPageRoute(builder: (context) => const CreateScreen()),
    );

    setState(() {});
  },
  child: const Icon(Icons.add),
),
);
}
}

```

todo.dart : 리스트에 보여지는 항목들-----

```

import 'package:hive_flutter/hive_flutter.dart'; //간단한 내용 저장 패키지

part 'todo.g.dart';

@HiveType(typeId: 0)
class Todo extends HiveObject {
  @HiveField(0)
  int? id;

  @HiveField(1)
  String title;

  @HiveField(2)
  int dateTime;

  @HiveField(3)
  bool isDone;

  Todo({
    this.id,
    required this.title,
    required this.dateTime,
    this.isDone = false,
  });
}

```

todo.g.dart : 리스트에 보여지는 항목들의 타입 -----

```
// GENERATED CODE - DO NOT MODIFY BY HAND
```

```
part of 'todo.dart';    //todo.dart 소스의 일부임을 명시
```

```
//part 키워드는 하나의 소스 파일의 코드를 여러 파일로 분할해서 저장할때 사용  
//library 키워드로 메인 소스에서 라이브러리를 선언하고 사용할 수 있지만 위와 같이 library 선언없이 사용가능  
//분할된 소스코드에서는 part of 키워드를 사용하여 메인 소스의 일부임을 명시해야 함.(part of main.dart)
```

```
// *****  
// TypeAdapterGenerator  
// *****
```

```
class TodoAdapter extends TypeAdapter<Todo> {  
  @override  
  final int typeId = 0;  
  
  @override  
  Todo read(BinaryReader reader) {  
    final numOfFields = reader.readByte();  
    final fields = <int, dynamic>{  
      for (int i = 0; i < numOfFields; i++) reader.readByte(): reader.read(),  
    };  
    return Todo(  
      id: fields[0] as int?,  
      title: fields[1] as String,  
      dateTime: fields[2] as int,  
      isDone: fields[3] as bool,  
    );  
  }  
  
  @override  
  void write(BinaryWriter writer, Todo obj) {  
    writer  
      ..writeByte(4)  
      ..writeByte(0)  
      ..write(obj.id)  
      ..writeByte(1)  
      ..write(obj.title)
```

```

        ..writeByte(2)
        ..write(obj.dateTime)
        ..writeByte(3)
        ..write(obj.isDone);
    }

    @override
    int get hashCode => typeId.hashCode;

    @override
    bool operator ==(Object other) =>
        identical(this, other) ||
        other is TodoAdapter &&
            runtimeType == other.runtimeType &&
            typeId == other.typeId;
}

```

todo_item.dart : 리스트 항목 보여주기, 항목 클릭했을 때 삭제 가능-----

```

import 'package:flutter/material.dart';
import 'package:intl/intl.dart';

import 'todo.dart';

class TodoItem extends StatelessWidget {
    final Todo todo;
    final Function(Todo) onTap;
    final Function(Todo) onDelete;

    const TodoItem({
        super.key,
        required this.todo,
        required this.onTap,
        required this.onDelete,
    });

    @override
    Widget build(BuildContext context) {
        return ListTile(
            onTap: () {
                onTap(todo);
            },
            trailing: IconButton(
                icon: Icon(Icons.delete),
                onPressed: () {
                    onDelete(todo);
                },
            ),
        );
    }
}

```

```

    },
    leading: todo.isDone
      ? const Icon(Icons.check_circle, color: Colors.green)
      : const Icon(Icons.check_circle_outline),
    title: Text(
      todo.title,
      style: TextStyle(color: todo.isDone ? Colors.grey : Colors.black),
    ),
    subtitle: Text(
      DateFormat.yMMMd()
        .format(DateTime.fromMillisecondsSinceEpoch(todo.dateTime)),
      style: TextStyle(color: todo.isDone ? Colors.grey : Colors.black),
    ),
    trailing: todo.isDone
      ? GestureDetector(
          onTap: () {
            onDelete(todo);
          },
          child: const Icon(Icons.delete_forever),
        )
      : null,
  );
}
}

```